

B.2 ELABORACIÓN DE LIBRO

DOCUMENTO(S) QUE DEBE INTEGRAR LA PROPUESTA DE TRABAJO.

a) Formato para la propuesta de trabajo para elaboración de libro

Instituto, Unidad o Centro: Tecnológico Nacional de México campus Tijuana	
Nombre del (de la) docente: José Mario García Valdez	
*Disciplina (s) (Rama(s) o campo(s) de estudio) [Remitirse al catálogo anexo]: Computación	
Nombre y clave del programa educativo al que mayormente impacta (plan de estudios): Ingeniería en Inteligencia Artificial (IIAR-2024-248)	
Título del Libro: Python para Análisis de Datos, Computación Inteligente, Cómputo Distribuido y Servicios Web.	
Justificación académica (exposición de motivos específicos) del programa a realizar, especificando los beneficios que obtiene el tecnológico o centro y el TecNM con los apuntes a desarrollar (una cuartilla): Durante mi año sabático, desarrollaré un libro sobre Python con el objetivo de proporcionar una guía y referencia para estudiantes y profesionales con conocimientos previos en programación que desean actualizarse en este lenguaje, especialmente en aplicaciones para ingeniería y ciencias de la computación. El libro abarcará conceptos de programación avanzada y el uso de librerías para desarrollar aplicaciones de vanguardia en control inteligente, aprendizaje automático, cómputo evolutivo distribuido y desarrollo de servicios web. Los temas del libro abarcan conceptos de programación avanzada y el uso de módulos (librerías) para desarrollar aplicaciones de vanguardia en áreas como control inteligente, aprendizaje automático, cómputo evolutivo distribuido y desarrollo de servicios web. A diferencia de otros textos o apuntes de materias específicas, este libro organiza y presenta de manera cohesiva contenidos temáticos de diversas asignaturas del plan de estudios de la maestría en ciencias computacionales, así como de los programas más recientes a nivel ingeniería en ingeniería en sistemas, informática, inteligencia artificial y ciencia de datos. Así, estudiantes, profesores y profesionales interesados en estos temas contarán con un recurso didáctico actualizado, con ejemplos prácticos que podrán usar como guía, referencia o inspiración en sus proyectos. Además, el libro incluirá temas para el despliegue de proyectos como servicios web e integración de soluciones tecnológicas avanzadas con Python, lo cual podría incentivar la creación de aplicaciones innovadoras y la formación de empresas de servicios tecnológicos.	
MARQUE EL PERIODO QUE CORRESPONDA () SEMESTRE 1er Periodo:	Desarrollar el objetivo general del libro: El lenguaje Python se ha consolidado como una herramienta esencial en la industria, especialmente en áreas de cómputo científico como la inteligencia artificial, ciencia de datos y minería de datos. Existen distribuciones del lenguaje que incluyen módulos integrados específicamente para estos fines, y su integración con servicios en la nube permite el análisis

20-01-2025 AI 19-04-2025 (X) AÑO 1er Periodo: 20-01-2025 AI 19-07-2025	de datos mediante dichos recursos. Este libro tiene como objetivo general presentar una guía completa de programación en Python dirigida a profesionistas y estudiantes de las áreas de ingeniería y ciencias computacionales, que ya cuentan con conocimientos de programación, pero desean utilizar el lenguaje de Python para realizar proyectos de computación inteligente, análisis de datos o sistemas distribuidos. Capítulo 1: Introducción a Python Objetivo: Familiarizar al lector con la instalación y configuración de Python en distintos sistemas operativos, incluyendo el uso de Jupyter Notebooks como herramienta interactiva de programación y otras herramientas similares. Secciones: Instalación Básica, Múltiples versiones, Distribuciones de Python, Jupyter Notebooks y Google Colaboratory. Avance breve: El lenguaje Python es un lenguaje interpretado, de propósito general, de código abierto y multiparadigma que fue diseñado originalmente por el desarrollador holandés Guido van Rossum a principios de los años noventa. La intención de Guido era crear un lenguaje de scripts, fácil de programar y que fuera legible, por lo que se utilizan sangrías para dar legibilidad al código y estas forman parte del lenguaje. De hecho, el nombre del lenguaje es en honor al grupo inglés de comedia Monty Python. Veamos un programa sencillo en Python para observar más fácilmente las diferencias: <pre>s = "Hola" x = 23 y = 12.78 if x > 14 or y < 20: print(x) else: print(y) x= " Mundo" print(x + s)</pre> A diferencia de lenguajes como C# o Java, podemos observar que las variables no incluyen una especificación del tipo de dato. Referencias: Van Rossum, G. (2003). <i>An introduction to Python</i> (p. 115). F. L. Drake (Ed.). Bristol: Network Theory Ltd.. Lutz, M. (2013). <i>Learning python: Powerful object-oriented programming</i>. " O'Reilly Media, Inc." Downey, A., Wentworth, P., Elkner, J., & Meyers, C. (2016). <i>How to think like a computer scientist: learning with python 3</i>. Muruges, T. S., Vasudevan, S. K., & Pulari, S. R. (2024). <i>Python: A Practical Learning Approach</i>. CRC Press. Jalolov, T. S. (2023). <i>TEACHING THE BASICS OF PYTHON PROGRAMMING</i>. International Multidisciplinary Journal for Research & Development, 10(11).
--	---

Capítulo 2: Fundamentos del Lenguaje Python

Objetivo: Introducir los conceptos básicos del lenguaje Python, cubriendo estructuras de datos fundamentales.

Secciones: Introducción al lenguaje, Estructuras (Secuenciales, Diccionarios, Conjuntos)

Avance breve:

Python incluye estructuras de datos secuenciales para almacenar objetos de manera similar a los arreglos o listas dinámicas en otros lenguajes. De manera general, les llamamos **objetos tipo secuencia** a las estructuras que podemos recorrer secuencialmente y en las cuales también podemos recuperar objetos almacenados en ellas utilizando índices. Los objetos tipo secuencia más utilizados son:

Tuplas

Las tuplas almacenan a una secuencia indizada e **inmutable** de ítems. Los ítems almacenados en la tupla pueden estar atados a cualquier tipo de objetos, incluso a otras colecciones. La característica de ser inmutable significa que, una vez generada la tupla ésta no se puede modificar, no podemos insertar ni borrar sus ítems.

Cadenas (Strings)

Representan una secuencia, también **inmutable**, de caracteres. Conceptualmente, son similares a las tuplas, pero estos objetos incluyen métodos particulares para operar sobre el texto.

Listas

Las listas son secuencias **mutable** de objetos; al ser mutable tiene métodos adicionales para realizar operaciones de gestión de los ítems contenidos en ellas.

Las palabras claves en estas estructuras son la mutabilidad en inmutabilidad de las estructuras. Veamos con detalle estas características.

Referencias:

Van Rossum, G. (2003). *An introduction to Python* (p. 115). F. L. Drake (Ed.). Bristol: Network Theory Ltd..

Downey, A., Wentworth, P., Elkner, J., & Meyers, C. (2016). *How to think like a computer scientist: learning with python 3*.

Capítulo 3: Manipulación de Cadenas de Texto

Objetivo: Enseñar técnicas para trabajar con cadenas de texto en Python, incluyendo métodos comunes y manipulación avanzada.

Secciones: Cadenas de Texto

Avance breve:

La clase **String** contiene varios métodos muy útiles para dar formato a las cadenas de texto. Estos métodos se ejecutan directamente en los objetos tipo cadena. Por ejemplo, para convertir una cadena a mayúsculas, podemos directamente llamar al método miembro **upper** de esta manera:

```
>>> "hola mundo".upper( )  
"HOLA MUNDO"
```

Referencias:

Van Rossum, G. (2003). *An introduction to Python (p. 115)*. F. L. Drake (Ed.). Bristol: Network Theory Ltd..

Capítulo 4: Programación Funcional

Objetivo: Explorar los principios de la programación funcional en Python, incluyendo funciones de orden superior y expresiones lambda.

Secciones: Programación Funcional, map reduce, cerradura, zip

En Python las funciones son, dicho de manera coloquial, ciudadanas de primera clase. Esto significa que fácilmente podemos utilizarlas como cualquier objeto: enviarlas como parámetros a otras funciones, regresarlas como resultado de otra función y almacenarlas en un diccionario u secuencia. Veamos como primer ejemplo la función **map** incluida en la librería estándar.

La función **map** toma dos parámetros: como primer parámetro toma una función y como segundo parámetro una lista. La función aplica la función que recibe a todos los elementos de la lista y como resultado nos regresa un objeto iterable, que contiene como elementos los objetos resultantes de aplicar la función. Veamos un ejemplo:

```
>>> lista = ['1','2','3']  
>>> map(int,lista)  
[1, 2, 3]
```

Referencias:

Downey, A., Wentworth, P., Elkner, J., & Meyers, C. (2016). *How to think like a computer scientist: learning with python 3*.

Lott, S. F. (2018). *Functional Python programming: Discover the power of functional programming, generator functions, lazy evaluation, the built-in itertools library, and monads*. Packt Publishing Ltd.

Capítulo 5: Manejo de Archivos

Objetivo: Proporcionar las herramientas para leer, escribir y manipular archivos en Python.

Secciones: Archivos, escritura, manipulación de directorios

Avance breve:

A diferencia de otros lenguajes, en Python podemos leer un archivo de texto con una sola línea de código. El método **open** es parte de la librería estándar. Aunque no es una solución para producción, ya que no considera la gestión de excepciones, podemos abrir y recorrer un archivo de texto de la siguiente manera:

```
datos = open("datos.txt")
for línea in datos:
    print(línea)
datos.close()
```

Referencias:

Van Rossum, G. (2003). *An introduction to Python* (p. 115). F. L. Drake (Ed.). Bristol: Network Theory Ltd..

Lutz, M. (2013). *Learning python: Powerful object-oriented programming*. " O'Reilly Media, Inc."

Capítulo 6: Programación Orientada a Objetos (OOP)

Objetivo: Introducir los principios de OOP en Python, cubriendo clases, herencia y encapsulación.

Secciones: Programación Orientada a Objetos, Herencia, Polimorfismo

Avance breve:

Hasta el momento hemos utilizado el lenguaje de Python como un lenguaje de scripts. Pero si ponemos atención, descubrimos que hemos estado utilizando objetos todo el tiempo. Las estructuras, las cadenas de texto, los diccionarios, los tipos básicos como enteros y flotantes también son objetos:

```
>>> "hello".upper()
>>> lista_nombres.append("Ana")
>>> diccionario.keys()
```

Aunque utilizamos objetos, haciendo llamadas a sus métodos, también podemos utilizar clases y herencia para definir nuevos tipos de objetos. Es importante mencionar que Python no implementa el paradigma orientado a objetos de una manera tan integrada como otros lenguajes orientados a objetos puros como C# o Ruby. Algunos de los elementos no se integran tan bien y parecería que la funcionalidad se agregó después y tomando atajos. Veamos a continuación un ejemplo implementado la clase **Estudiante**:

```
class Estudiante:
    """Documentación de la clase"""
    def __init__(self, numero_control, nombre):
        self.numero_control = numero_control
        self.nombre = nombre
```

Referencias:

Van Rossum, G. (2003). An introduction to Python (p. 115). F. L. Drake (Ed.). Bristol: Network Theory Ltd..

Lutz, M. (2013). Learning python: Powerful object-oriented programming. " O'Reilly Media, Inc."

Capítulo 7: Módulos y Librerías de Python

Objetivo: Explorar módulos estándar y librerías de terceros que amplían las capacidades de Python.

Secciones: Módulos estándar, Otros módulos, Requests, Beautiful Soup

Avance breve:

Para hacer peticiones HTTP podemos utilizar la librería estándar **httplib** que incluye clases para hacer peticiones HTTP y HTTPS, pero es recomendable utilizar para nuestros proyectos la librería Requests que incluye clases de más alto nivel de abstracción. Veamos este ejemplo que encontramos en su documentación:

```
r = requests.get('https://miapi.com/usuario, auth=('user', 'pass'))
>>> r.status_code
200
>>> r.headers['content-type']
'application/json; charset=utf8'
>>> r.encoding
'utf-8'
>>> r.json()
{'capitulos_publicados': 19, 'articulos': 77, ...}
```

Referencias:

Hajba, G. L. (2018). *Website Scraping with Python*. Berkeley: Apress.

Mitchel, R. (2024). *Web Scraping with Python: Data Extraction from the Modern Web*. O'Reilly Media.

Hetland, M. L., & Nelli, F. (2024). *Network Programming*. In *Beginning Python: From Novice to Professional* (pp. 309-324). Berkeley, CA: Apress.

Capítulo 8: Manejo de Ambientes Virtuales

Objetivo: Introducir la creación y gestión de ambientes virtuales para proyectos Python.

Secciones: Ambientes Virtuales, PyEnv

Avance breve:

Ahora que hemos visto cómo instalar módulos y librerías completas para utilizar en nuestros proyectos, vemos que puede ser un problema mantener al día las versiones de estas librerías, sobre todo si son desarrolladas por terceros. Para un proyecto podríamos utilizar una versión y en otro probablemente una versión más reciente. Para evitar este tipo de problemas, podemos crear instalaciones completas de Python aisladas unas de otras, cada una con sus propios módulos. En Python le llamamos ambientes virtuales a estas instalaciones. Es recomendable crear un ambiente virtual para cada proyecto independiente que hagamos, de esta manera no mezclamos versiones. Un ambiente puede incluir incluso distintas versiones del intérprete de Python, no solo de las librerías de terceros.

Referencias:

Van Rossum, G. (2003). *An introduction to Python* (p. 115). F. L. Drake (Ed.). Bristol: Network Theory Ltd..

Lutz, M. (2013). *Learning python: Powerful object-oriented programming*. " O'Reilly Media, Inc."

Capítulo 9: Bases de Datos Relacionales en Python

Objetivo: Proveer al lector los conocimientos necesarios para trabajar con bases de datos SQL.

Secciones: SQLite, PostgreSQL, ORM

Avance breve:

SQL es un gran lenguaje de consulta y de gestión de bases de datos relacionales. SQL es un lenguaje declarativo y por alguna razón existe la noción de que es más difícil de utilizar que lenguajes imperativos como Python. Además de esto, cada sistema gestor de base de datos define su propio dialecto de SQL intentando agregar funcionalidades propias. Esto hace difícil migrar bases de datos de un sistema a otro sin tener que hacer modificaciones a las consultas o la definición del esquema.

Por otro lado, los lenguajes orientados a objetos tienen la capacidad de representar datos de una forma más natural, ya que cada registro de una tabla puede ser representado como un objeto y cada columna como un atributo. Un ORM (Object-Relational Mapping) es una técnica de programación que nos permite operar con bases de datos relacionales utilizando objetos en lugar de SQL. Los ORMs permiten a los desarrolladores de software, utilizar el mismo lenguaje para la implementación de la lógica de negocio y la interacción con la base de datos. El uso de un ORM, es entonces una decisión de diseño que dependerá de las necesidades del proyecto.

Django es un framework diseñado principalmente para crear aplicaciones web que utilizan bases de datos relacionales para la gestión de la información. Por lo tanto, un componente importante es su ORM. En otra sección vamos con mucho mayor detalle el uso de Django, en esta sección solo nos concentraremos en el API de Django para realizar las operaciones de CRUD (Create, Read, Update, Delete). Para esto, utilizaremos el esquema de la base de datos (modelo en términos del framework) de la aplicación de **MyMovies** que definimos anteriormente:

Capítulo 10: Bases de Datos NoSQL

Objetivo: Introducir los conceptos y herramientas para trabajar con bases de datos NoSQL.

Secciones: Redis, MongoDB, S3

Avance breve:

Creación de Buckets en AWS S3

Ahora vamos a crear un **bucket** de almacenamiento desde la terminal **bash**. Para esto, reemplaza el nombre <nombre-bucket> especificando el nombre que deseas asignarle a tu propio **bucket**:

```
aws s3api create-bucket --bucket <nombre-bucket> --region us-east-1
```

	Algo importante es que debemos elegir un nombre que no exista actualmente dentro de AWS por lo que se recomienda utilizar el dominio de nuestra aplicación o algo similar. La terminal te debe regresar una confirmación de que el bucket ha sido creado. Ahora vamos a confirmar en la consola de gestión de S3 que el bucket se ha creado correctamente. Inicialmente, el bucket se crea con el permiso Block all public access, con lo cual se bloquea el acceso público. Para controlar mejor este tipo de permisos, vamos a crear un archivo de políticas para el bucket que acabamos de crear. El archivo lo llamaremos <code>website_security_policy.json</code>. Debemos reemplazar <code><bucket-name></code> por el nombre que le dimos anteriormente y en la dirección IP vas a poner la dirección externa de tu computadora local. En caso de que no conozcas esta dirección IP, puedes utilizar un servicio como https://www.whatismyip.com/. Referencias Kumar, A. (2022). <i>Using Redis for persistent storage in serverless architecture to maintain state management</i> (Doctoral dissertation, Dublin, National College of Ireland). Da Silva, M. D., & Tavares, H. L. (2015). <i>Redis Essentials</i>. Packt Publishing Ltd. Douglas, K., & Douglas, S. (2003). <i>PostgreSQL: a comprehensive guide to building, programming, and administering PostgreSQL databases</i>. SAMS publishing.
Contará con 30 días naturales para la entrega del reporte intermedio (50%) ante el TecNM a partir de la fecha:	MARCAR ☐ Para Semestre 19 de abril de 2025 ☒ Para Año 19 de julio de 2025
MARQUE EL PERIODO QUE CORRESPONDA ☐ SEMESTRE 2do Periodo: 20-04-2025 AI 19-07-2025 ☒ AÑO 2do Periodo: 20-07-2025 AI 19-01-2026	Capítulo 11: Análisis de Datos con Python Objetivo: Capacitar al lector en el uso de librerías para análisis y visualización de datos. • Secciones: Numpy, Pandas, Matplotlib • Avance breve: <u>Pandas</u> es una biblioteca para gestión y análisis de datos en Python. La biblioteca nos ofrece estructuras de datos flexibles y rápidas para el análisis de datos. Según su documentación, pandas nos permite la gestión de varios tipos de datos: • Series de tiempo, las cuales pueden o no estar ordenadas. • Tablas con columnas heterogéneas. • Matrices homogéneas o heterogéneas. Las clases principales que implementan esta funcionalidad son Series y DataFrame. La estructura Series trabaja con vectores, que nos sirven para representar series de tiempo. Por otro lado, están los DataFrames que utilizamos para aquellos datos que tienen una estructura de tabla o matriz. En esta sección nos enfocaremos en la estructura DataFrame, ya que en este caso nos interesa trabajar con datos organizados como tablas. Referencias

Erfina, A., & Nurul, M. R. (2023). *Implementation of Naive Bayes classification algorithm for Twitter user sentiment analysis on ChatGPT using Python programming language*. Data and Metadata, 2, 45-45.

Zulunov, R., & Soliev, B. (2023). *Importance of Python language in development of artificial intelligence*. Потомки Аль-Фаргани, 1(1), 7-12.

Landau, R. H., Pérez, M. J., & Bordeianu, C. C. (2024). *Computational physics: Problem solving with Python*. John Wiley & Sons.

McKinney, W. (2017). *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython (2nd ed.)*. O'Reilly Media.

Grus, J. (2019). *Data Science from Scratch: First Principles with Python (2nd ed.)*. O'Reilly Media.

Capítulo 12: Introducción al Aprendizaje Automático

Objetivo: Enseñar los fundamentos de aprendizaje automático, desde el preprocesamiento hasta los algoritmos de aprendizaje automático como árboles de decisión o redes neuronales.

- **Secciones:** Scikit-Learn, Preprocesamiento, Redes Neuronales
- **Avance breve:** Ejemplos de entrenamiento y evaluación de modelos.

Para este tipo de texto vamos a utilizar un codificador de tipo Tf-Idf el cual utilizamos anteriormente. Al llevar a cabo este análisis de textos, nos interesa mantener como parte del texto elementos propios de los mensajes, como son: los emoticones, emojis, texto en mayúsculas y signos de puntuación. Para realizar el procesamiento de este ejemplo, necesitamos importar la librería **emoji** la cual nos permite cambiar los emojis a su versión en texto y viceversa, utilizando los métodos emojiize y demojiz:

```
>>> print(emoji.emojiize('Python is :thumbs_up:'))
```

Python is 👍

```
>>> print(emoji.demojize('Python is 👍'))
```

Python is :thumbs_up:

Instalamos la biblioteca en nuestro ambiente virtual utilizando PIP:

```
pip install emoji
```

Referencias

Subasi, A. (2020). *Practical machine learning for data analysis using python*. Academic Press.

Raschka, S., Patterson, J., & Nolet, C. (2020). *Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence*. Information, 11(4), 193.

Chollet, F. (2021). *Deep learning with Python*. Simon and Schuster.

Gallatin, K., & Albon, C. (2023). *Machine learning with Python cookbook*. " O'Reilly Media, Inc."

Capítulo 13: Computación Inteligente

Objetivo: Introducir técnicas avanzadas de inteligencia computacional, como lógica difusa y simulaciones.

- Secciones: Lógica Difusa (Scikit Fuzzy), Simulación (ODE), Control Difuso
- Avance breve: Ejemplos de lógica difusa y simulaciones.
- Ilustraciones: Representaciones gráficas de sistemas difusos.

Propuesta por Lofti Zadeh, la lógica difusa introduce los conceptos de conjuntos difusos y operadores lógicos difusos. A diferencia de la lógica booleana, en la cual un elemento es miembro de un conjunto o no lo es, en la lógica difusa los elementos tienen un grado de pertenencia (o membresía) a múltiples conjuntos. La lógica difusa utiliza las llamadas funciones de membresía (MFs, por sus siglas en inglés) para asignar un valor numérico a cada miembro del conjunto, indicando su grado de pertenencia. Las MFs deben definirse para cada variable lingüística; estas variables pueden utilizarse en un sistema basado en reglas para expresar conocimientos de manera similar al lenguaje natural. Por ejemplo, la regla “**si la distancia es cercana**” utiliza la variable lingüística **distancia**, con una MF difusa que asigna un grado de pertenencia al conjunto **Cercano** para cada elemento del **dominio** de distancia; por ejemplo, para 2mm y 50mm una función podría asignar los siguientes grados de pertenencia: **cercano(2)=0.92**, como los valores de membresía van de cero a uno, en este caso 2mm tiene un alto valor de membresía al conjunto cercano. Ahora, para 50mm la función de membresía nos regresa: **cercano(50)=0.40**, esto significa que su valor de membresía es menor. Estos valores de membresía nos pueden parecer adecuados para ciertos problemas, pero para otros no. Es por eso que debemos especificar para cada aplicación los valores que consideramos adecuados para cada dominio en particular. Estos sistemas basados en reglas pueden expresar relaciones complejas, adecuadas para aplicaciones de control.

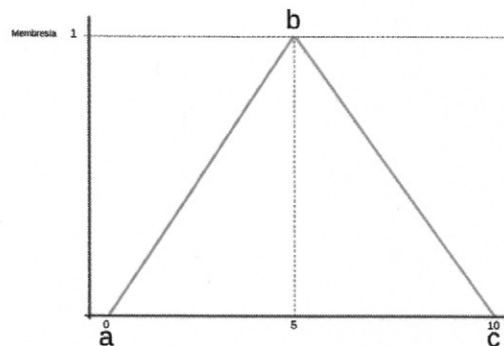
(Explicamos con mayor detalle la lógica difusa y después pasamos a los ejemplos de programación)

El paquete **SciKit-Fuzzy** incluye una colección de algoritmos de lógica difusa para **SciPy** que es un software especializado en matemáticas, ciencia e ingeniería. **SciKit-Fuzzy** utiliza arreglos de **numpy** por lo que también debe estar instalado. Podemos importar los dos paquetes con las siguientes instrucciones:

```
>>> import numpy as np
>>> import skfuzzy as fuzz
```

En este caso utilizamos los alias **np** y **fuzz** que comúnmente se utilizan para que los nombres sean más cortos. Un componente importante de la lógica difusa son las funciones: las variables difusas y las funciones de membresía.

Podemos crear una función de membresía triangular como la que se muestra en la figura:



Como primer paso, debemos definir el dominio de la función, por ejemplo, queremos que el dominio vaya de cero a diez:

```
>>> dominio = np.arange(11)
```

Ahora vamos a establecer los parámetros de la función de membresía triangular, elegimos los valores $a=0$, $b=5$ y $c=10$. Estos valores se especifican en un arreglo como vemos a continuación.

```
>>> fm_triangular = fuzz.trimf(x, [0, 5, 10])
```

Podemos ver el dominio:

```
>>> dominio
```

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

Para ver los grados de membresía en IPython en el dominio especificado, desplegamos el contenido del objeto `fm_triangular`:

```
>>> fm_triangular
```

```
array([ 0. ,  0.2,  0.4,  0.6,  0.8,  1. ,
        0.8,  0.6,  0.4,  0.2,  0. ])
```

Referencias

Gupta, K., Kumar, P., Upadhyaya, S., Poriye, M., & Aggarwal, S. (2024). *Fuzzy logic and machine learning integration: Enhancing healthcare decision-making*. *International Journal of Computer Information Systems and Industrial Management Applications*, 16(3), 20-20.

Fumanal-Idocin, J., & Andreu-Perez, J. (2024). *Ex-Fuzzy: A library for symbolic explainable AI through fuzzy logic programming*. *Neurocomputing*, 128048.

Razak, T. R., Ul-Saufie, A. Z., Yusoff, M. H., Ismail, M. H., Fauzi, S. S. M., & Zaki, N. A. M. *Python scikit-fuzzy: developing a fuzzy expert system for diabetes diagnosis*. *Int J Artif Intell ISSN*, 2252(8938), 1399.

Ikegwu, A. C., Nweke, H. F., & Anikwe, C. V. (2024). *Recent trends in computational intelligence for educational big data analysis*. *Iran Journal of Computer Science*, 7(1), 103-129.

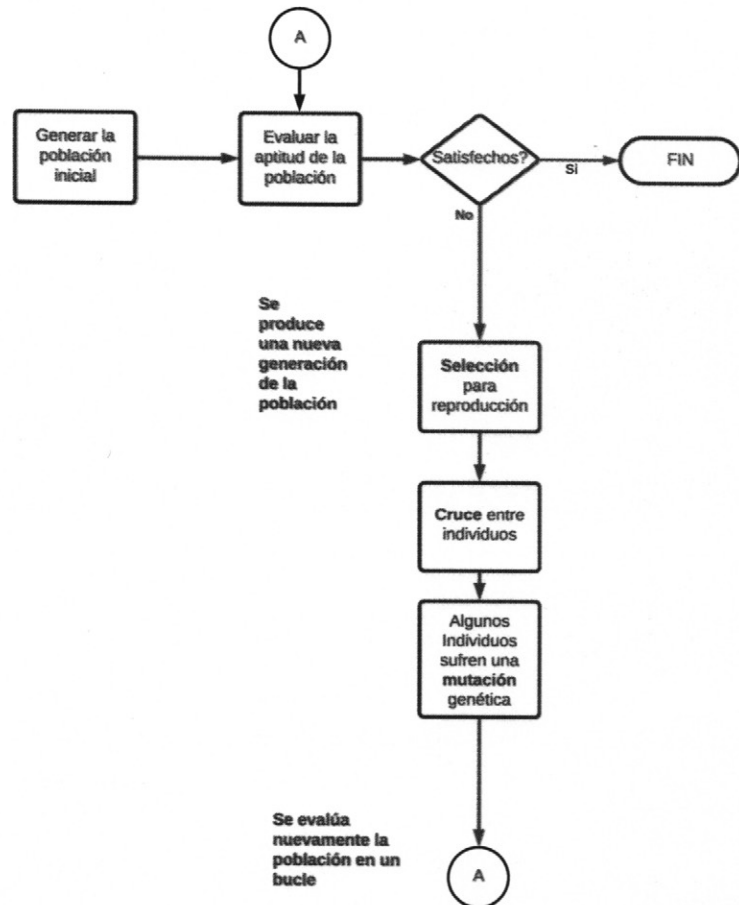
Capítulo 14: Cómputo Evolutivo

Objetivo: Explorar métodos de optimización evolutiva como algoritmos genéticos y PSO.

- Secciones: GA, PSO
- Avance breve:

Las metaheurísticas basadas en poblaciones son técnicas estocásticas que buscan soluciones óptimas utilizando dos estrategias: la exploración y la explotación. La exploración lleva a cabo una búsqueda global en el espacio de soluciones, evitando quedar atrapada en óptimos locales, mientras que la explotación hace una búsqueda local en torno a soluciones prometedoras cercanas. Los **Algoritmos Genéticos** (GAs por sus siglas en inglés) son los representantes canónicos de las metaheurísticas basadas en poblaciones. Los GAs operan sobre un conjunto de posibles soluciones, denominado población, que evoluciona a lo largo de un cierto número de generaciones hasta encontrar una solución adecuada u óptima. En cada generación, se evalúan las posibles soluciones (individuos), y luego los individuos seleccionados se reproducen mediante operadores de cruce genético (exploración) y de

mutación (explotación) para generar nuevos descendientes. Finalmente, un mecanismo de reemplazo selecciona a los descendientes más adaptados, los cuales se utilizan para formar una nueva generación de la población. La mayoría de las metaheurísticas basadas en poblaciones crean un conjunto inicial de soluciones candidatas aleatorias, que luego se evalúan mediante una función de aptitud. Tomando en cuenta la calidad y la posición o estructura de cada solución, se aplica una **metaheurística** inspirada en la naturaleza para generar un nuevo conjunto de candidatos. Por ejemplo, cada solución se puede representar como una lista de valores y se puede generar una nueva solución con un operador de cruce. Este operador puede generar una nueva lista, incluyendo algunos elementos de las ambas listas. El diagrama de flujo de un GA básico puede representarse de esta manera:



Referencias

De Rainville, F. M., Fortin, F. A., Gardner, M. A., Parizeau, M., & Gagné, C. (2012, July). *Deap: A python framework for evolutionary algorithms*. In Proceedings of the 14th annual conference companion on Genetic and evolutionary computation (pp. 85-92).

Gad, A. F. (2024). *Pygad: An intuitive genetic algorithm python library*. Multimedia tools and applications, 83(20), 58029-58042.

Objetivo: Introducir los principios de computación distribuida basada en contenedores, nativa en la nube y serverless. Se incluyen ejemplos prácticos de cómo implementarlos en Python.

- **Avance breve:**

La computación en la nube se está convirtiendo en un estándar para realizar experimentos en ciencias de la computación. Este cambio se atribuye en gran medida a su modelo rentable de pago por uso, que elimina la necesidad de realizar grandes inversiones iniciales y los costos continuos de gestión asociados con infraestructuras locales. Además, la computación en la nube facilita la definición de la infraestructura dentro del propio código, mejorando enormemente la reproducibilidad de los resultados en computación científica. Un ejemplo básico de los beneficios de tener un código fácilmente reproducible es el auge de plataformas de computación interactiva basadas en la web, como el proyecto [Jupyter Notebook](#) o [Google Colaboratory](#).

Al ejecutar código interactivo en estas plataformas, es posible elegir fácilmente escalar el entorno de ejecución, utilizando recursos computacionales que superan considerablemente los de nuestras computadoras personales. Esta capacidad es una ventaja clave, ya que permite realizar operaciones más complejas e intensivas en recursos, superando las limitaciones del hardware local. Una implementación nativa en la nube de un experimento computacional también permite escalar desde una ejecución local a una amplia gama de opciones de ejecución mediante servicios de computación en la nube, como AWS de Amazon o Azure de Microsoft.

La computación en la nube ha evolucionado, pasando de alojar aplicaciones monolíticas en máquinas virtuales que se ejecutan en centros de datos remotos a otras arquitecturas. En la actualidad, los patrones de diseño predominantes en la computación en la nube siguen los principios de microservicios y *serverless*. La arquitectura de microservicios enfatiza la descomposición de las aplicaciones en componentes más pequeños y acoplados de manera flexible, lo cual fomenta la escalabilidad y flexibilidad. Por otro lado, la computación *serverless* lleva este paradigma un paso más allá al abstraer la infraestructura, permitiendo a los desarrolladores centrarse únicamente en la ejecución del código y ajustando dinámicamente la asignación de recursos para satisfacer las demandas de la aplicación.

El uso combinado de estos patrones de diseño mejora la eficiencia y adaptabilidad de los experimentos computacionales en entornos nativos de la nube.

Referencias:

Dainamang, S. A., Praherdhiono, H., & Soepriyanto, Y. (2024). *Design of learning python programming for informatics education student using cloud computing technology based on google colaboratory*. Journal of Research in Instructional, 4(1), 111-120.

Capítulo 16: Desarrollo de Servicios Web

Objetivo: Proporcionar los conocimientos para desarrollar y desplegar servicios web usando Python.

- **Secciones:** FastAPI, Django
- **Avance breve:**

En el contexto del framework de Django, la clase Form es una representación de un formulario HTML, y nos permite crear y gestionar la información de dentro de los formularios HTML de manera programática. Los formularios de Django se especifican en una clase que hereda de `django.forms.Form`. Por ejemplo, vamos a definir un formulario para agregar comentarios:

	<pre>class CommentForm(forms.Form): name = forms.CharField(label="Nombre") url = forms.URLField(label="Tu website", required=False) comment = forms.CharField()</pre> De manera similar a los modelos de Django, Form también genera código HTML considerando los campos y las validaciones especificados en la clase. En este caso, por cada campo que necesitamos en el formulario, vamos a definir un atributo en nuestra propia clase CommentForm. Cada campo es a su vez una clase que hereda de Field la cual nos sirve para validar y gestionar los datos ingresados por el usuario. La documentación de Django provee una lista de los campos disponibles en el módulo django.forms.fields y sus respectivos atributos y métodos. (FormFields)[docs.djangoproject.com/en/5.0/ref/forms/fields/]. En nuestro ejemplo, el formulario tiene tres campos: el nombre del usuario, la URL de su sitio web y un comentario. En el caso de la URL, utilizamos la clase URLField, y especificamos en el constructor que el campo es opcional mediante el parámetro required=False y la etiqueta del campo se especifica con el parámetro label. Una ventaja importante es que el objeto url puede validar además que la URL ingresada sea válida. Referencias: Dainamang, S. A., Praherdhiono, H., & Soepriyanto, Y. (2024). <i>Design of learning python programming for informatics education student using cloud computing technology based on google colaboratory</i>. Journal of Research in Instructional, 4(1), 111-120.	
Contará con 30 días naturales para la entrega del reporte final (100%) ante el TecNM a partir de la fecha:	MARCAR ☐ Para Semestre 19 de julio de 2025 ☒ Para Año 19 de enero de 2026	
Firma del (de la) docente: 		
Fecha de elaboración: (dd/mm/aaaa) 05/11/2024		

El (la) docente deberá presentar como propuesta el desarrollo de una disciplina (Rama o campo de estudio) del área de conocimiento afín a su perfil profesional y ejercicio docente.

Área	Disciplina
Educación	Educación
Administración – Industrial	Administración - Económico-Administrativo Contador Público Ingeniería Administrativa Planificación - Gestión - Logística - Desarrollo Comunitario - Manufactura
Agropecuaria	Acuicultura Agricultura Agroecosistemas Ambiental Biotecnología y Alimentos Forestal Pecuaria
Arquitectura	Arquitectura Ingeniería Civil Construcción
Eléctrica - Electrónica - Mecatrónica – Mecánica	Ingeniería Eléctrica Ingeniería Electromecánica Ingeniería Electrónica Ingeniería Mecánica Ingeniería Mecatrónica Ingeniería en Metalurgia Ingeniería en Materiales
Química	Química Bioquímica Alimentos Electroquímica Nanotecnología
Tecnologías de la Información y Comunicación	Computación Informática TIC's

- b) Oficio aval del Departamento de Desarrollo Académico, el cual se debe tramitar ante el Departamento de Desarrollo Académico
- c) Incluir reporte de software especializado en el que se muestre que no se tiene un porcentaje mayor al 30% de similitud, con el visto bueno de la Subdirección Académica. (Indicar el nombre del titular de ésta).

La documentación que conforma la propuesta de trabajo deberá escanearse en un solo archivo en formato PDF con un tamaño máximo de 2MB.